

MAX 26.27.1 V8 — Технический аудит мода

Версия стока: 26.27.1 (build 6802) **Версия мода:** V8 (Privacy Mod by JohNick) **Дата:** 2026-08-24

Совместимость: ru.one.me.app (Android 8.0–16.0 / API 26–36) · arm64-v8a + armeabi-v7a · основной + клон (ru.one.me.ap2)

> **V8:** первый релиз объединённой линии разработки — сток V7 (портирован полностью) + сквозное E2E-шифрование личных чатов (Signal Protocol) + два новых фикса. Ветки 26.27.1 GP и 26.27.1 GP e2e консолидированы: 26.27.1 GP e2e стала каноническим dev-репозиторием.

□ 26.27.1 V8 — что нового

□ Сквозное шифрование личных чатов (E2E)

Реализация на **Signal Protocol** (X3DH для установления сессии + Double Ratchet для forward secrecy на каждом сообщении). Ключи никогда не покидают устройство. Сервер MAX видит только шифротекст — сам протокол устойчив к компрометации сервера.

Активация — из меню личного чата (пункт «**Начать защищённый чат**»). Обмен pre-key bundle между устройствами — по QR-коду или deeplink. Верификация собеседника — сверкой отпечатка (fingerprint) в меню чата.

Подробная справка для пользователя — прямо в приложении: «□ **ПРИВАТ** → **E2E Шифрование**» (шаги подключения, что делать при рассинхроне, как сверить отпечатки).

Компоненты (все в отдельных smali-классах мода one/me/e2e/*, не пересекающихся со стоком):

- Крипто-ядро: X3DH-handshake, Double-Ratchet-state, HKDF-цепочки, message key derivation.
- Persistence: ключи в encrypted SharedPreferences, one-time pre-keys ротируются автоматически.
- Deeplink-bootstrap: обработка max-e2e://... для авто-подтверждения ключей.
- UI-слой: QR-скан, отображение fingerprint, статус защищённости в тулбаре чата.

Каналы и групповые чаты E2E пока не поддерживают (только 1-на-1). Сообщения, отправленные **до** активации E2E, остаются в исходном формате — новые шифруются.

Device-verified на HA218XJZ (Lenovo, 2026-08-24): wire-шифрование подтверждено, 31/31 unit-тест крипто-ядра PASS.

□ Фикс «Вид → Выбрать обои чата» — авто-включение прозрачности

Регресс V6/V7: пикер обоев (AppFlags.pickWallpaper() → системный picker → Modswallpaper.saveFromUri() → mods_wallpaper.jpg) успешно сохранял выбранное изображение, но показ картинки в ChatScreen шёл через Modswallpaper.apply(ViewGroup), а этот метод гейтился хелпером apply_v_wallpaper_gate.py (V6) на флаг AppFlags.transparentTheme:Z. При выключенной «Прозрачной теме» файл сохранялся, но визуально не появлялся — пользователю казалось, что кнопка не работает.

Фикс: при выборе новых обоев через pickWallpaper() автоматически включается transparentTheme (или transparentThemeChats — в зависимости от активной комбинации), после чего ChatScreen пересоздаётся (recreate()) и обои сразу видны без ручной настройки тумблера. Sentinel: # :wallpaper_picker_autotransparent_v1.

□ Фикс «повторно всплывают уже просмотренные реакции» при «Невидимке» — reaction-ledger

Регресс, вылезший при активной моде-античиталке: одна и та же реакция после холодного рестарта снова показывалась как непрочитанная. Первопричина установлена A/B-тестом (invisible OFF → нет re-surface; invisible ON → есть): мод-хук в c3/a5e.smali::c(JJJZZZ)V (штатный funnel «reaction seen») блокировался при invisible/antiRead, чтобы сервер не узнал о просмотре. Как следствие — локальный маркер прочитанности реакции не сохранялся между рестартами, а бейдж в списке чатов лит на sw2.k0 без сравнения с seen-маркером.

Фикс — независимый ledger `rr_<chatId>` в `SharedPreferences("app_flags")`, зеркальный по логике уже существующему ledger'y прочитанных сообщений (`r1_<chatId>`). Три хелпера в `AppFlags` + WRITE-хук в `c2/qy6.smali` (`MarkReactionAsReadUseCase`) + GATE-хук в `c1/nh3.smali` (рендер строки списка):

- `AppFlags.rrRecord(JJ)V` — при просмотре реакции пишет `rr_<chatId> = lastReactedMessageId` (exact overwrite).
- `AppFlags.rrCaughtUp(JJ)Z` — точное совпадение `stored == j0`.
- `AppFlags.gateReaction(Lsw2;)Z` — комплексный гейт: если `unreadGateActive(chatId) && j0>0 && rrCaughtUp(a, j0) → true` (подавить бейдж).
- WRITE-хук в `qy6.smali` перед вызовом `La5e;->c(JJJZZZZ)V: invoke-static {v12, v13, v5, v6}, Lone/me/util/AppFlags;->rrRecord(JJ)V`.
- GATE-хук в `nh3.smali: iget-object v2, v1, Lys2;->b:Lsw2; / invoke-static {v2}, ...->gateReaction(Lsw2;)Z` → если `true`, `v9 = 0` (флаг бейдж-lit сбрасывается).

Sentinels: `# :reaction_ledger_record (qy6)`, `# :reaction_ledger_gate (nh3)`, `# :gateReaction (AppFlags)`.

Механическая корректность подтверждена независимым адверсариальным верификатором (статический анализ smali): `msgid (v5:v6)` в `qy6 = Lg3f;->a:J = Lsw2;->j0:J = Protos$Chat.lastReactedMessageId` — то же самое поле, что читает `gateReaction`. Сам оригинальный код `qy6` логирует `v5:v6` как `"msgid="`, `v12:v13` как `"chatsid="` — семантика аргументов совпадает.

«Невидимка» продолжает работать: `read-mark` на сервер не уходит, но локально бейдж больше не всплывает после рестарта.

□ Унаследовано из 26.27.1 V7 (без изменений)

□ «Прозрачный фон в чатах» — независимый тумблер

До V7 обои внутри открытого чата показывались только при общей «Прозрачной теме». В V7 добавлен отдельный флаг `AppFlags.transparentThemeChats:Z` (тумблер «□ Прозрачный фон в чатах» в категории «□ ВИД»), делающий прозрачным **только внутренность чата**, не трогая список/звонки/настройки.

Динамическое управление фоном контейнера по жизненному циклу `ChatScreen`:

- `AppFlags.applyRootTransparency(View, I)` в `fqe.onThemeChanged` красит корневой `fqe` и контейнер по режиму.
- `ChatScreen.onAttach → AppFlags.chatContainerAttach(View): chat-only → контейнер 0x7f090939` прозрачный, инкремент `chatBgDepth`.
- `ChatScreen.onDetach → AppFlags.chatContainerDetach(View): декремент, возврат к списку — контейнер снова непрозрачный.`

Оконная прозрачность (`FLAG_SHOW_WALLPAPER + translucent theme`) в `MainActivity.onCreate` включается при **любом** из двух флагов — OR-гейт в `applyTransparentIfEnabled` и `applyTransparentThemeEarly`.

Гейт обоев внутри чата (`Modswallpaper.apply + t93.setBackground`) переведён на `transparentThemeChats`. Обои на списке чатов (`applyAsBackground`) — под `transparentTheme`.

Sentinels: `# :root_transparency_helper_v1`, `# :chat_lifecycle_bg_v1`, `# :root_transparent_bg_v3`.

□ Пикатель вложений в альбомном режиме — фикс layout

`ChatScreen.k2 (open-hook): setRequestedOrientation(PORTRAIT = 0x1)`. `jb.h:pswitch_31 (close-hook): восстановление USER = 0x2`.

□ Унаследовано из 26.27.1 V6 (без изменений)

□ Обои чата на списке чатов

`Modswallpaper.applyAsBackground(View)V` ставит `BitmapDrawable (Gravity.FILL)` на `root-View` `ChatsTabWidget.onCreateView`. Sentinel `# :c1_wallpaper_bg_v1`.

❑ «Прозрачная тема» — гейт обоев списка

Флаг `AppFlags.transparentTheme:Z` (default `false`). Sentinel # `:wp_gate_v1`.

❑ Кнопки тем «Свет / Тёмная / AMOLED»

`ContextWrapper.getBaseContext()`-unwrap-loop до реальной `Activity` + `recreate()`. Sentinel `:ocv_theme_unwrap_loop`.

❑ Унаследовано из 26.27.1 V5 (без изменений)

❑ Альбомный/горизонтальный режим восстановлен

В `MainActivity.y(Ljava/lang/Boolean;)V` форсируется `const/4 v3, 0x1` → `setRequestedOrientation(USER)`.

❑ Лимит закреплённых чатов снят по умолчанию

`unlockPinLimit=true` В `AppFlags.<clinit>; pinLimit(I)I` → `Integer.MAX_VALUE`. Тумблер `hidden`.

❑ Ограничение серверное: сервер может сбросить лишние пины при синхронизации.

❑ Унаследовано из 26.27.1 V4 (без изменений)

❑ PIN-защита чата

`ChatPinManager.hideView(View)` → `INVISIBLE (WeakReference)`; `revealView()` только на верном PIN; `dropHidden()` + `finishActivity()` на неверном. Минимум 4 цифры.

❑ Порядок закреплённых чатов после drag

`PinDragCb.n()`: инверсия `if-eqz` → `if-nez`. Порядок — `AppFlags.putString("pinned_order", csv)`.

❑ «Удерживать связь в фоне» (`bgWatchdog`)

Настройки → Звонки и система. По умолчанию ВКЛЮЧЁН. Escape-hatch для MIUI/HyperOS.

❑ Унаследовано из более ранних версий (без изменений)

- ❑ Chat-PIN (V2/V3): `locked_chats_csv` + `ChatPinReceiver` + long-press заголовка.
- ❑ Drag ACTION_DOWN consume (V3): `PinHandleTouch.onTouch` → `true`.
- ❑ Гудок дозвона (V2): `stop()` перед `Connection.destroy()`.
- ❑ Прокрутка чата под невидимкой (V2): якоря `bta/Lv2f;->g:J`.
- ❑ Нейтрализация телеметрии (V2): `push`, `aggregation`, `operator-fetch (IP-strip)`, `DPS`.
- ❑ Обход App Lock (V1): `onStart`-гейт + `appLockGateActive:Z`.
- ❑ «О приложении» → настройки мода (V1): long-press.
- ❑ Визуальная метка удалённых (26.26.0 V5): красный крестик + `setAlpha 0.5f`.
- ❑ Форс-консьюм при открытии чата (26.26.0 V5): гейт `smartAntiRead || antiRead`.

❑ Профиль приватности

Невидимка (`invisible`) — мастер-переключатель

Дочерний флаг	Что делает
<code>antiTyping</code>	собеседник не видит набор текста
<code>antiRead</code>	собеседник не видит отметку «прочитано»

Дочерний флаг	Что делает
smartAntiRead	«прочитано» уходит только после вашего ответа
antiPresence	онлайн / last-seen скрыты

Reaction-ledger (V8, новый)

rr_<chatId> в SharedPreferences("app_flags"). Активен при том же условии, что и r1_<chatId> (invisible OR (antiRead && !smartAntiRead) OR (smartAntiRead && chat opened)). Не изменяет серверное поведение — только локально подавляет re-surface бейджа.

E2E (V8, новый)

Signal Protocol X3DH + Double Ratchet. Ключи никогда не покидают устройство, сервер видит только шифротекст. Отдельный от стандартной серверной защиты MAX слой.

Антиудаление (antiDelete) — 4 слоя

1. **L3** — Central WS chat-event. 2. **L4** — Room ORM body-clear. 3. **Маркер** ☐ + время удаления. 4. **FCM офлайн-персист**.

Нейтрализация телеметрии

Чекпоинты в push-транспорте, aggregation-логах, operator-fetch (IP-strip), DPS-пинге.

☐ Верификация (V8)

Независимая адверсариальная кросс-верификация — статический анализ smali по 3 DEX собранного MAX_26.27.1_arm64_V8.apk (baksmali) + device-verify на S25 и HA218XJZ:

Проверка	Результат
E2E крипто-ядро: X3DH-handshake, Double-Ratchet-state, HKDF-цепочки (31/31 unit-rect)	☐ PASS
E2E wire-verified на HA218XJZ: шифротекст на wire, дешифровка на устройстве	☐ PASS (2026-08-24)
Reaction-ledger: AppFlags.rrRecord/rrCaughtUp/gateReaction присутствуют, сигнатуры совпадают	☐ подтверждено
Reaction-ledger: WRITE-хук в c2/zy6.smali перед La5e; ->c, sentinel # :reaction_ledger_record	☐ подтверждено
Reaction-ledger: GATE-хук в c1/nh3.smali, sentinel # :reaction_ledger_gate	☐ подтверждено
Reaction-ledger MATCH: msgid (v5:v6) в zy6 = Lg3f; ->a:J = Lsw2; ->j0:J = Protos\$Chat.lastReactedMessageId (независимый агент-верификатор)	☐ MATCH
Wallpaper-picker auto-transparent: AppFlags.pickWallpaper включает transparentTheme перед recreate(), sentinel # :wallpaper_picker_autotransparent_v1	☐ подтверждено
AppFlags V7-хелперы (transparentThemeChats, rootThemeColor, chatBgDepth, applyRootTransparency, chatContainerAttach/Detach)	☐ подтверждено
ChatScreen.onAttach/onDetach: chatContainerAttach/Detach после invoke-super (sentinel # :chat_lifecycle_bg_v1)	☐ подтверждено
fqe.onThemeChanged: applyRootTransparency(View, I)V вместо setBackgroundColor (sentinel # :root_transparent_bg_v3)	☐ подтверждено
Оконная прозрачность OR-гейт `transparentTheme \	\
Пикатель вложений: ChatScreen.k2 lock PORTRAIT, jb.h:pswitch_31 restore USER	☐ подтверждено
MainActivity.y(): const/4 v3, 0x1 (auto-rotate USER)	☐ подтверждено (V5)

Проверка	Результат
AppFlags.pinLimit(): unlockPinLimit=true → Integer.MAX_VALUE	☐ подтверждено (V5)
ChatPinManager: hideView / revealView / dropHidden / abortHidden	☐ подтверждено (V4)
Регистровый аудит apply-скриптов по классам коллизий	☐ 0 COLLISION
apksigner verify (все 4 APK)	☐ v2+v3, exact ABI, distinct SHA
Regression-smoke стоковых V7-функций на V8 (S25, main+clone): cold-start, «Настройки мода» → все 7 категорий, экран «ВИД» (обои/темы/акценты/прозрачность), контекст-меню сообщения (RedCode), папки	☐ PASS

☐ Артефакты (V8)

Файл	SHA-256
MAX_26.27.1_arm64_V8.apk	589a227b92a75606814af632880a2d2eae55aa667ca719f7bf1354f02a0c85d
MAX_26.27.1_arm64_V8_clone.apk	000822b4b5c1ee3dc71c92e4b355c9d3a918c3918e5d9bb89ebe9118e23b6d96
MAX_26.27.1_arm7_V8.apk	4c9b414598b29efda89a7ee8c99d367a4fd8b68ed434e31e52e9d481361e8e63
MAX_26.27.1_arm7_V8_clone.apk	209c3726e5eef42c4a3240ce7bf7c47241357aa510cb24499e0e7f0d37ebc5e3

Подпись: v2+v3 APK Signature Scheme. Cert SHA-1 идентичен всей линейке 26.x — `install -r` без потери данных.

Аудит выполнен статическим анализом *smali* (*baksmali* + независимый агент-верификатор) по декомпиляции фактически собранных APK. Динамический *smoke* — на S25 (RFCX5013V5F) и HA218XJZ (Lenovo, PIN 1982) через adb: E2E wire-verified (24.08), reaction-ledger runtime-verified (24.08), regression-smoke стоковых V7-функций PASS (24.08).